

Designing Tools Claude Picks Correctly

Descriptions as interface, when to split and when to merge, and why a tool that works is not the same as a tool that gets used properly.

Claude never sees your code. The **description** is the entire interface.

Anthropic's five principles

The frame for this whole module.

- ◆ **Choose the right tools.** A few high-leverage tools beat a thin wrapper around every API endpoint.
- ◆ **Namespace them.** Clear, distinct names so the agent is not guessing between neighbours.
- ◆ **Return meaningful context.** Human-readable fields, not raw identifiers the agent must look up.
- ◆ **Optimise for tokens.** Defaults, filtering, pagination, truncation.
- ◆ **Prompt-engineer the description.** It is a prompt, so write it like one.

ON THE EXAM

Every question in this module is one of these five wearing a scenario.

What belongs in a tool description

- ◆ **What it does**, in one clear sentence.
- ◆ **When to use it**, in terms of the situation rather than the mechanism.
- ◆ **When not to use it**, and which tool to reach for instead.
- ◆ **What it returns**, so the agent can plan the next step before calling.
- ◆ **Flag destructive or irreversible behaviour explicitly**. The agent cannot infer it from the name.

ON THE EXAM

"When not to use this" is the line most descriptions are missing, and it is the one that stops wrong selection.

The same tool, described two ways

DESCRIPTION	WHAT THE AGENT DOES WITH IT
"Searches the database."	Calls it for everything, including things it cannot answer.
"Searches customer order history by customer ID. Returns orders from the last 24 months. Use lookup_order for a single known order number. Does not cover cancelled orders."	Calls it when it has a customer ID and wants history, and reaches elsewhere otherwise.

ON THE EXAM

Same code. Same schema. Completely different behaviour, because the second one answers "when" and "when not".

Names and namespaces

- ◆ **Distinct names beat clever names.** `get_user` and `fetch_user` in the same toolset is a coin flip.
- ◆ **Namespace by domain** when you have several sources. `finance_get_expenses` says where it lives.
- ◆ **Overlapping tools are a selection problem**, not a documentation problem. Fix the overlap.
- ◆ **Name the parameter unambiguously.** Anthropic's own example: `user_id`, not `user`.

ON THE EXAM

If two tools could plausibly answer the same request, the agent will sometimes pick the wrong one. That is a design fault.

Parameters that stop guessing

- ◆ **Enumerate what you can.** An enum makes an invalid value impossible rather than unlikely.
- ◆ **Describe each field**, including format. "start_date, ISO 8601, e.g. 2026-09-01".
- ◆ **Required means required.** Anything the agent might not have should be optional with a sensible default.
- ◆ **A required field the agent cannot know is an invitation to fabricate one.**
- ◆ **Enforce with strict data models**, so an ambiguous input fails loudly rather than quietly.

ON THE EXAM

Fabricated arguments are almost always a schema problem. The agent had to put something in the box.

A tool is a contract between a
deterministic system and a **non-**
deterministic user.

Split the tool, or merge them?

SPLIT

One tool doing several jobs

- A mode parameter selects between behaviours
- The description has to say "or" a lot
- The agent picks the wrong mode sometimes
- Different jobs need different permissions

MERGE

Several tools always called together

- The agent chains three calls to get one answer
- Each result is only useful as input to the next
- Intermediate results waste context
- A dropped step breaks the sequence silently

THE TIEBREAKER

Ask what a single successful use looks like. If it is always three calls, merge. If it is one of three behaviours, split.

Resolve identifiers inside the tool

- ◆ **The chain:** the agent calls `search_customer`, reads an ID out, then calls `get_orders` with it.
- ◆ **Every hop is a turn**, a round trip, and an ID sitting in context doing nothing useful.
- ◆ **Better:** `get_orders` accepts an email or a name, and does the lookup internally.
- ◆ **Return the readable field too**, so the agent can talk about "Acme Ltd" rather than a UUID.

ON THE EXAM

Make the tool take what the agent has, not what your database wants.

Designing what comes back

- ◆ **Structured data, not formatted prose.** The agent parses it; nobody is reading your table borders.
- ◆ **Human-readable fields alongside IDs,** so no follow-up call is needed to say what happened.
- ◆ **Only what is needed.** A `response_format` or `detail` parameter lets the agent ask for more.
- ◆ **Returning everything is not generous.** It is spending the agent's context on your behalf.

ON THE EXAM

Claude Code truncates tool responses at 25,000 tokens by default. That default exists for a reason.

Searching for a contact by reading every page from the top is not thoroughness. It is **brute force**.

Token efficiency in practice

- ◆ **Sensible default limits.** Fifty results, not everything, unless asked.
- ◆ **Pagination, ranges and filters,** so a big result set can be approached in pieces.
- ◆ **Truncate with instructions,** telling the agent how to fetch the continuation.
- ◆ **Steer towards many small searches** rather than one broad one, in the description itself.

ON THE EXAM

Truncating without saying how to continue turns a partial answer into a wrong answer.

Tool definitions are not free

- ◆ **Every definition is input tokens on every single request.** Not once. Every turn.
- ◆ **Large MCP toolsets can consume tens of thousands of tokens** before the agent has read the user's question.
- ◆ **Selection quality degrades as the toolset grows.** More neighbours means more chances to pick wrong.
- ◆ **A few well-chosen tools outperform many overlapping ones**, on both cost and accuracy.

ON THE EXAM

This is why "add another tool" is so often the wrong answer. Every addition taxes every request.

Currency check: too many tools

TOPIC	EXAM ANSWER	CURRENT REALITY
Large toolsets	Consolidate. Fewer, better tools.	Still true, and the Tool Search Tool now loads definitions on demand.
Context cost	All definitions are loaded upfront.	Deferred loading keeps only what the task needs.
The design principle	A few high-leverage tools.	Unchanged. Search does not excuse a sprawling toolset.

ON THE EXAM

Answer "consolidate" on the exam. Know that on-demand tool loading exists so the docs do not surprise you.

Do not port your API

- ◆ **Endpoints are organised around your data model.** Tools should be organised around the agent's tasks.
- ◆ **A thin wrapper per endpoint** produces forty tools, heavy overlap and constant chaining.
- ◆ **Start from a real workflow** and build the few tools that workflow needs.
- ◆ **Prototype, test on realistic tasks, refine.** Three to five cases is enough to see a large effect.

ON THE EXAM

The larger the effect size, the smaller the sample you need to see it. Start with a handful of real tasks.

Reading the stem: tool design questions

WHAT THE STEM SAYS	WHERE TO LOOK
"calls the wrong tool for this kind of request"	Description, and overlap between neighbours
"passes values that do not exist"	Schema. A required field it cannot know.
"three calls to answer one question"	Merge, or resolve the ID inside the tool
"picks the wrong mode"	Split. One tool is doing several jobs.
"context fills up after a few tool calls"	Response design. Filter, paginate, truncate.
"never uses the tool at all"	Description does not describe a situation it recognises

ON THE EXAM

Almost every symptom here points at the description or the schema, not at the prompt.

Module 5 in five lines

- ◆ The description is the interface. Say what it does, when to use it, when not to, and what comes back.
- ◆ Fabricated arguments are a schema fault. Never require what the agent cannot know.
- ◆ Merge when success is always three calls. Split when one tool has three behaviours.
- ◆ Design the response. Structured, filtered, readable, and truncated with instructions.
- ◆ Every tool definition taxes every request, in tokens and in selection accuracy.

ON THE EXAM

Seven worked scenarios next. Two of them are the split-or-merge decision pointing in opposite directions.

A support agent has two tools. `lookup_order` is described as "Retrieves order information." `search_orders` is described as "Searches orders."

In production the agent frequently calls `search_orders` when the customer has quoted a specific order number, then filters the results itself. It occasionally calls `lookup_order` with a customer email, which fails.

The team's proposal is to add guidance to the system prompt explaining which tool to use when.

What is the most effective fix?

- ☐ A Add the guidance to the system prompt as proposed
- ☐ B Rewrite both descriptions to state what each takes, what each returns, and when to use the other
- ☐ C Merge the two tools into one that accepts either an order number or a search query
- ☐ D Remove `search_orders` so only one tool remains

◆ PAUSE AND DECIDE

What is the most effective fix?

- A

 System prompt guidance
Puts selection rules far from the point of selection. It also has to be repeated for every future tool.
- B

 Rewrite both descriptions
The descriptions never said what each accepts. That is the information the agent was missing.
- C

 Merge into one tool
Two genuinely different jobs with different inputs. Merging creates the mode problem you would then have to split.
- D

 Delete search_orders
Removes a capability the agent legitimately needs when no order number is known.

THE SIGNAL

Both descriptions are quoted, and neither mentions inputs. The stem is showing you the gap.

An extraction agent has a tool `create_ticket` with required parameters: title, description, priority, assignee_id and department_code.

Users report tickets appearing with plausible but incorrect assignees, and department codes that do not exist. The agent has no access to the staff directory or the department list.

The title and description are consistently accurate.

What is the root cause?

- ☐ A The model is hallucinating and needs a lower temperature
- ☐ B assignee_id and department_code are required but unknowable, so the agent invents values
- ☐ C The tool description does not explain what a department code is
- ☐ D The agent needs a system prompt instruction never to guess parameter values

◆ PAUSE AND DECIDE

What is the root cause?

- A

 Lower the temperature
Makes the invented values more consistent, not more correct. The agent still has to fill the field.
- B

 Required but unknowable
A required field the agent has no source for leaves it one option: produce something schema-valid.
- C

 Description does not explain
Explaining what a code is does not tell it which code. The information is absent, not unclear.
- D

 Instruct it not to guess
The schema says required. An instruction not to guess and a schema demanding a value are in direct conflict.

THE SIGNAL

"Title and description are consistently accurate" is the control. The two failing fields are exactly the two it has no source for.

A property research agent answers questions about listings. To answer "what is the council tax band for the flat on Mill Road?" it currently calls `search_properties` to find the property, reads the property ID from the result, calls `get_property` to fetch details, reads the local authority ID, then calls `get_tax_band`.

Three round trips per question. Traces show the intermediate IDs are never used for anything else.

What is the most effective change?

- ☐ A Run the three calls in parallel to reduce latency
- ☐ B Provide a tool that takes a property description or address and returns the tax band, resolving IDs internally
- ☐ C Cache the intermediate IDs so repeated questions skip the lookups
- ☐ D Add a system prompt example showing the correct three-call sequence

◆ PAUSE AND DECIDE

What is the most effective change?

- A

 Run them in parallel
They cannot run in parallel. Each call needs an ID produced by the previous one. This is a chain.
- B

 One tool, internal resolution
Success is always the same three calls, so the sequence belongs inside the tool.
- C

 Cache the IDs
Helps repeat questions about the same property. The first question about any property still takes three hops.
- D

 Show the sequence
The agent is already performing the sequence correctly. Teaching it the thing it does is not a fix.

THE SIGNAL

"The intermediate IDs are never used for anything else" is the merge signal. They exist only to feed the next call.

A document tool is defined as `manage_document`, with a required action parameter accepting "read", "update", "delete" and "share", plus fifteen other parameters used by different actions.

Two problems recur. The agent sometimes calls it with action "update" when the user only asked a question. And the security team cannot grant read access without also granting delete.

What is the most appropriate change?

- ☐ A Split it into separate tools, one per action, each with only its own parameters
- ☐ B Add a confirmation parameter that must be set to true for update and delete
- ☐ C Improve the description to explain when each action should be used
- ☐ D Add a hook that blocks delete actions the user did not ask for

◆ PAUSE AND DECIDE

What is the most appropriate change?

- A

 Split by action
Fixes both symptoms at once. Distinct tools are selected more reliably, and permissions become per-tool.
- B

 Confirmation parameter
The agent sets that flag too. A parameter the model controls is not a control.
- C

 Better description
May reduce wrong-mode selection. Does nothing at all about the permissions problem.
- D

 Hook on delete
Real enforcement, aimed at one action. The stem's problem is update, and the permission issue remains.

THE SIGNAL

Two symptoms, one cause. When a permissions requirement cannot be expressed, the tool boundary is in the wrong place.

A codebase agent has a tool `list_files` that returns every file path in the repository. The repository has around eight hundred files.

Engineers report that after two or three tool calls the agent starts forgetting details from earlier in the conversation, and its answers become vaguer.

The team is considering moving to a model with a larger context window.

What should you recommend?

- ☐ A Move to the larger context window as planned
- ☐ B Replace `list_files` with a search tool that accepts a pattern and returns matching paths, with a default limit
- ☐ C Have the agent summarise the file list after receiving it
- ☐ D Call `list_files` once at the start and instruct the agent to reuse the result

◆ PAUSE AND DECIDE

What should you recommend?

- A

 Larger context window
Buys headroom and changes nothing. The tool still returns eight hundred paths on every call.
- B

 Search with a default limit
The tool stops dumping. The agent gets the page it needs instead of the whole book.
- C

 Summarise after receiving
The full list has already entered context by then. Summarising adds tokens on top.
- D

 Call once and reuse
Better than calling repeatedly, and eight hundred paths still sit in context for the whole session.

THE SIGNAL

This is the address book. The fix is to skip to the right page, not to buy a bigger desk.

An integration exposes forty tools, generated automatically one per REST endpoint. Many overlap: `get_user`, `get_user_profile` and `fetch_user_details` all return overlapping data.

The agent frequently selects a tool that returns almost the right information, then calls another to get the missing field. Token costs are high even on simple requests.

One engineer proposes writing much more detailed descriptions for all forty.

What is the most effective approach?

- ☐ A Write the detailed descriptions, since the description is the interface
- ☐ B Identify the workflows the agent actually performs and build a small set of tools for them
- ☐ C Keep all forty but instruct the agent in the system prompt to prefer `get_user`
- ☐ D Split the forty tools across several MCP servers to reduce per-server load

◆ PAUSE AND DECIDE

What is the most effective approach?

- A

 Better descriptions for all forty
Descriptions cannot separate tools that genuinely overlap, and forty definitions still tax every request.
- B

 Build for the workflows
Fixes overlap, chaining and token cost together, because all three come from porting the API.
- C

 Prefer `get_user`
One hard-coded preference among forty overlapping tools. The next ambiguity is unaddressed.
- D

 Split across servers
All definitions still load into the same context. Reorganising the shelf does not shrink the library.

THE SIGNAL

Option A uses this module's own lesson as bait. Descriptions are the interface, and no description resolves genuine overlap.

A finance agent has a tool `get_transactions`. Its description reads: "Returns transactions. Set `include_all` to true for the complete history."

The agent sets `include_all` to true on nearly every call, retrieving several years of records to answer questions about last month. The tool has no default limit and no date filter.

Costs are four times the estimate.

What is the most effective change?

- ☐ A Instruct the agent in the system prompt to avoid setting `include_all` unless necessary
- ☐ B Add date range parameters and a sensible default limit, and describe when the full history is appropriate
- ☐ C Remove the `include_all` parameter entirely
- ☐ D Truncate the response to the most recent fifty transactions

◆ PAUSE AND DECIDE

What is the most effective change?

- A

 Instruct it to avoid include_all
The tool offers no alternative. Without a date filter, all is the only way to be sure of covering last month.
- B

 Date range, default limit, guidance
Gives the agent the narrow option it lacked, and a default that is right when it does not choose.
- C

 Remove include_all
Deletes a legitimate capability. Some questions do need the full history.
- D

 Truncate to fifty
Silent truncation. The agent believes it saw everything and answers confidently from a partial view.

THE SIGNAL

The agent is behaving rationally. The only lever it had was all or nothing, and it needed to be sure.